Movie Recommendation System

Devansh Kumar
2022152

Yash Goyal
2022588

Aryan
2022109

Kartikeya Malik
2022243

Abstract

In today's digital age, users are constantly faced with an overwhelming amount of content, particularly in the entertainment industry. With a vast library of movies available, it becomes increasingly difficult for individuals to find what suits their tastes. This project was inspired by the need to enhance user experience by developing a content-based movie recommender system. By focusing on features such as genres, cast, keywords, and plot descriptions, the system aims to provide more personalized and relevant movie suggestions. The project leverages machine learning techniques to ensure that the system is both accurate and scalable, helping users discover films tailored to their unique preferences. Ultimately, this recommender system will reduce the effort and time required for users to browse, making content discovery more intuitive and efficient.
GitHub - https://github.com/ygyashgoyal/Movie-Recommendation-System.git

1. Introduction

In an era of overwhelming movie options, finding relevant recommendations is essential for enhancing user experience. This project aims to develop a hybrid recommender system combining **content-based filtering**, which analyzes movie attributes like genres and cast, with **collaborative filtering**, which leverages user preferences to identify patterns. By integrating both approaches, the system ensures personalized and diverse recommendations, making it easier for users to discover movies they are likely to enjoy.

2. Literature Survey

2.1. A Comprehensive Movie Recommendation System: Collaborative and Content-Based Filtering Integration:

This research presents a refined strategy for enhancing movie recommendation systems by merging content-based filtering with diverse collaborative prediction methods. While content-based filtering focuses solely on item attributes, collaborative prediction incorporates user-item interactions to boost recommendation precision. The method capitalizes on the advantages of both approaches, addressing weaknesses in traditional systems. By emphasizing diversity in collaborative prediction, the model captures a broader range of user preferences, delivering more tailored and accurate suggestions. Experimental comparisons affirm the effectiveness of this hybrid approach, improving accuracy and user satisfaction.

2.2. Enhanced Content–Based Filtering Using Diverse Collaborative Prediction for Movie Recommendation:

This research introduces an advanced technique for improving movie recommendation systems by combining content-based filtering with diverse collaborative prediction methods. Traditional content-based filtering focuses on item features, while collaborative prediction boosts recommendation accuracy by considering user interactions with items. The proposed method merges these two approaches, overcoming the limitations of existing systems. By incorporating diversity in collaborative prediction, the model captures a broader range of preferences, leading to more personalized and accurate recommendations. Experimental results validate its effectiveness, improving accuracy and user satisfaction.

3. Dataset

3.1. Data Preprocessing

- **Loading the dataset[3]**: We first load two CSV files, movies.csv and credits.csv, and merge them based on the movie title

- **Data Cleaning**:
  - Extract relevant columns such as 'movie_id','title','overview','genres','keywords','cast','crew','vote_average'.
  - For fields that contain lists of these columns using functions like extract,

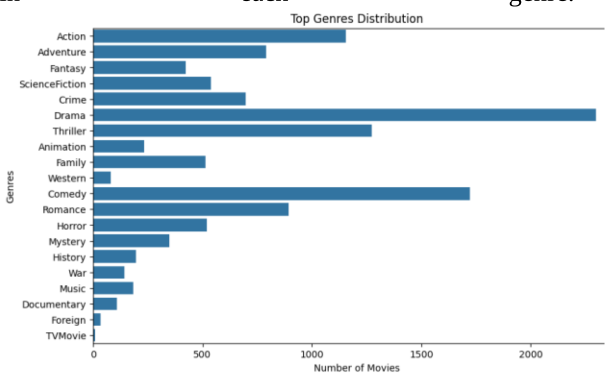
top_cast, and director to parse and clean these columns.

  - Null values are handled using dropna()[1].

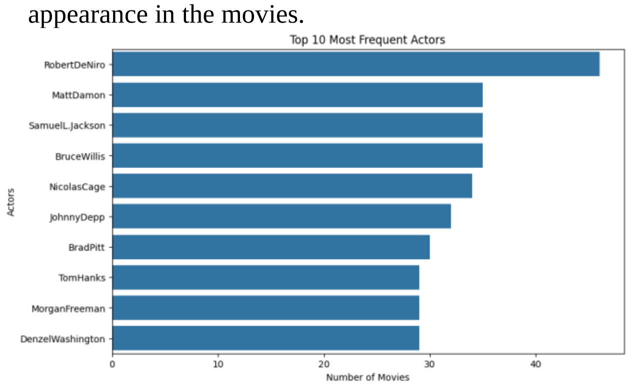
- **Text Processing**: Tokenization is performed to remove spaces from cast, crew, genres, and keywords to identify kind of likely entries.

3.2. Exploratory Data Analysis (EDA)

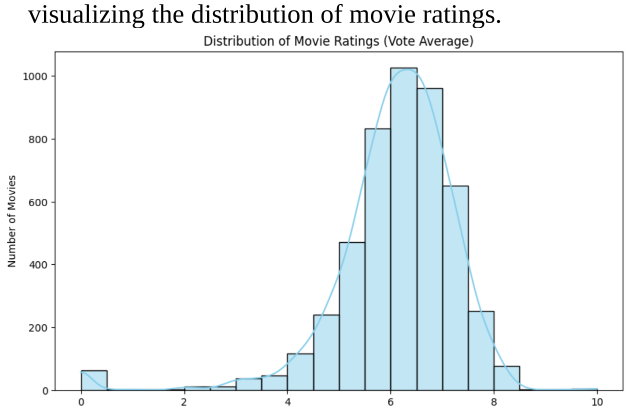
- We have visualized different aspects of your dataset:
- Genre Distribution: Used to plot the number of movies in each genre.



- Top 10 Actors: Shows the top 10 actors based on their appearance in the movies.



- Distribution of Movie Ratings: A histogram visualizing the distribution of movie ratings.



3.3. Structure of the Dataset Credits

- **movie_id**: This is a unique identifier for each movie in the dataset. For example, the movie ID is **19995** for "Avatar".
- **title**: This column contains the title of the movie.
- **cast**: This column contains details about the cast members. The information is typically stored as a string that represents an array of objects. Each object contains several attributes for individual cast members. Includes -: **cast_id**, **character**, **credit_id**, **gender, id**, **name**, **order**

3.4. Potential Uses for Credits dataset:

- Can be used to enhance our movie recommendation system by analyzing the cast members, their roles, and their relationships with other movies.
- Analyze patterns related to gender representation in films, leading roles, and the distribution of cast members across different movies.
- Extract features like the number of main characters, gender distribution among cast members, and character diversity.

3.5. Structure of the Dataset Movies

- **genres**: A JSON-formatted string containing genres associated with the movie.
- **keywords**: A JSON-formatted string listing keywords related to the movie's themes or plot.
- **overview**: A brief synopsis of the movie's plot.
- **title**: The title of the movie.
- **vote_average**: The average rating received by the movie.
- **vote_count**,**budget,** .**homepage**, **id**, **original_language**, **original_title**, **popularity**, **production_companies**, **production_countries**, .**release_date**, **revenue**, **runtime**, **spoken_languages**, **status**, **tagline.**

3.6. Potential Uses for This Data

- Extract and transform relevant features for your recommendation system, such as genres, keywords, and cast members.
- **Data Analysis**: Performed analyses on the correlation between budget and revenue or investigated how different genres affect popularity ratings.
- Used this information alongside the cast data to create a more comprehensive recommendation system that considers multiple factors.

4. Methodology and Model Details

4.1. Content-Based Filtering:

- Content-based filtering is implemented by leveraging movie-specific features such as genres, overview, cast, crew, and keywords to recommend movies similar to a given target movie.

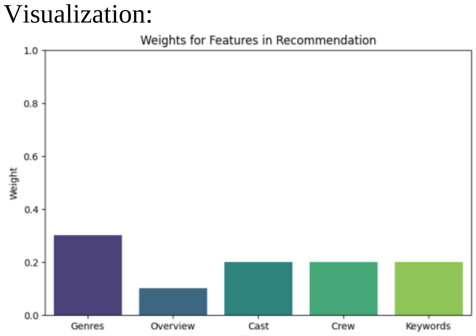
- **Feature Vectorization:**
  - Each feature (genres, overview, cast, crew, and keywords) is transformed into numerical vectors using **TF-IDF Vectorization**.
  - This step ensures that each movie is represented as a high-dimensional vector, capturing the importance of terms relative to the dataset while ignoring less informative words (stop words).
- **Cosine Similarity:**
  - For each feature vector, the **cosine similarity** is computed to measure the similarity between movies.
  - The similarity score for a feature S_f between two movies A and B is calculated as

$$S_f = \frac{\text{Dot Product of Vectors for } A \text{ and } B}{\text{Magnitude of Vector } A \times \text{Magnitude of Vector } B}$$

  - Scores range from 0 to 1, where values closer to 1 means high similarity, while scores closer to 0 indicate dissimilarity.
- **Weighted Aggregation:**
  - Individual similarity scores from each feature are aggregated using predefined weights (genres: 0.3, overview: 0.1, cast: 0.2, crew: 0.2, keywords: 0.2)
  - The final similarity score S is computed as:

$$S = \sum_f (W_f \times S_f)$$

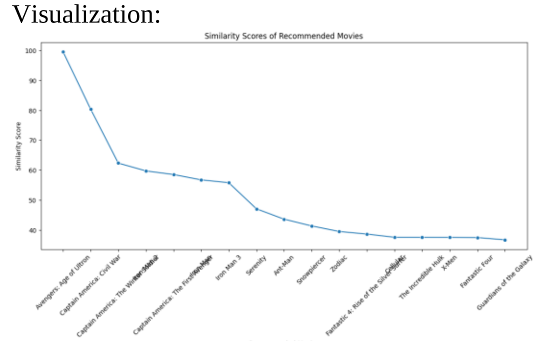
- **Recommendation Logic:**
  - For a given target movie, the final similarity scores are computed with all other movies in the dataset.
  - The scores are sorted in descending order, and the top N movies are returned as recommendations.
- **Flexibility in Features Weights:**
  - The system allows users to adjust the weights dynamically, enabling personalized recommendations tailored to user preferences.
  - Visualization:



- **Similarity Metric:**
  - The system generates similarities percentages using overlapping features to recommend movies with higher similarity.

4.2. Collaborative Filtering:

- Collaborative filtering is implemented to enhance the diversity of recommendations by considering user preferences and patterns.
- Collaborative Filtering also recognizes other users' data history and preferences to recommend similar content.
- **Similarity Calculation:**
  - For a target movie, similarity is computed using a combination of three metrics:
    - **Genre Overlap (Weight: 0.6)**: Measures the percentage overlap in genres between movies.
    - **Vote Average Similarity (Weight: 0.3)**: Compares the similarity in ratings (scaled between 0 and 1).
    - **Revenue Similarity (Weight: 0.1)**: Uses the difference in revenue values (normalized) to assess financial performance similarity.
  - The combined similarity score S_c is calculated as:
    - $S_c = (0.6 \times \text{Genre Overlap}) + (0.3 \times \text{Vote Average Similarity}) + (0.1 \times \text{Revenue Similarity})$
  - Visualization:



- **Top Matches:**
  - The similarity scores between the target movie and all other movies are computed and sorted in descending order.
  - The top N movies are selected based on the highest scores.
- **Integration with Content-Based Filtering:**
  - Collaborative filtering recommendations are blended with content-based results to provide users with a balanced mix of highly relevant and diverse recommendations.

4.3. Combined System Logic

- The recommendation system offers two modes:
  - **Content-Based Mode:** Focuses solely on the input movie's features.
  - **Collaborative Mode:** Broadens recommendations by incorporating user preference patterns.
- The flexibility in weights and combination allows the system to adapt to different recommendation scenarios, ensuring both relevance and diversity.

5. Results and Analysis:

5.1. Similarity Scores:

- **Functionality:** The algorithm generates movie recommendations by computing similarity scores based on shared features such as genres, keywords, and cast members**.**
- **Example:** For 'The Avengers':
  - Avengers: Age of Ultron; Rating: 7.3, Similarity Score: 99.56%, Matching Features: 3 genres, 6 keywords, 2 cast members
  - Captain America: Civil War; Rating: 7.1, Similarity Score: 80.36%, Matching Features: 3 genres, 6 keywords, 2 cast members

5.2. Analysis of Recommendations

- **Quality of Recommendations:**
  - The system effectively identifies movies closely related to the input (*The Avengers*), with the highest similarity scores reflecting the most relevant matches.
- **Diversity:**
  - Recommendations cover a range of similar movies, balancing genre alignment and diverse cast and keywords, with diminishing similarity scores broadening the suggestions.
  - The system's suggestion is also dependent upon other users' preferences.

5.3. A/B Testing

- **Methodology:**
  - A/B testing involves varying the weights assigned to specific features (e.g., genres, cast) to observe their impact on recommendation quality.
- **Insights**:
  - Preliminary observations suggest that fine-tuning feature weights can significantly enhance the relevance and diversity of the recommendations provided.

6. Conclusion

- This project further enhanced our understanding of building a movie recommendation system, with a focus on integrating and refining key techniques:
  - **Natural Language Processing (NLP):** Using techniques like **TF-IDF** and tokenization to convert textual data into numerical representations for machine learning models.
  - **Cosine Similarity:** Learned how to measure similarity between vectors to find related movies based on key features.
  - **Content-Based Filtering:** This iteration allowed us to improve the content-based recommendation by leveraging multiple features such as genres, keywords, and cast members to calculate similarity scores.
  - **Evaluation Metrics:** A new emphasis was placed on analyzing the recommendations using similarity scores and qualitative feedback to ensure both relevance and diversity.
  - **Tuning:** We also learned about tuning model weights for different features tailoring the recommendation quality to prioritize specific user preferences such as storyline over cast or genre.
  - **Enhanced Results Presentation:** Recommendations were presented with detailed insights, including similarity scores and matching features, offering more transparency and user engagement compared to the previous version.
  - **Collaborative Filtering:** This iteration allowed us to improve recommendations using similar user preferences.

- **Member's Contributions:**

  - Yash Goyal (2022588)-: Preprocessing, Model Implementation, Hyperparameter tuning, Performance Evaluation, Report

  - Devansh Kumar (2022152)-: Preprocessing, Model Implementation, Hyperparameter tuning, Performance Evaluation, Report

  - Aryan (2022109)-: Presentation

  - Kartikeya Malik (2022243)-: Presentation

References

[1] OpenAI. (2024). *ChatGPT* [Large language model]. https://chatgpt.com

[2] https://www.shiksha.com/online-courses/articles/movie-recommendation-system-using-machine-learning/

[3] https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata